



1. Что значит «обогащать» знания агента

Когда я говорю об обогащении информации, я имею в виду ситуацию, когда агент принимает решение не только на основании знаний самой языковой модели и текущего диалога. Мы подключаем к нему дополнительные источники, содержащие сведения о конкретном проекте, организации или предметной области.

Это принципиально важно, потому что модель может хорошо понимать Python, Git или архитектурные паттерны, но она не может заранее знать внутреннюю документацию нашей компании, текущую версию корпоративного API, правила конкретного проекта или состояние внутренней системы на сегодняшний день.

Поэтому качество агента определяется не только выбранной моделью. Оно сильно зависит от того, к каким внешним знаниям агент получает доступ и насколько правильно эти источники используются.

2. Статические и динамические источники

Дополнительную информацию полезно разделять по характеру её изменения. Часть знаний относительно стабильна. Например, архитектурное описание проекта, внутренний стандарт разработки или инструкция по развёртыванию могут изменяться достаточно редко.

Другие данные меняются постоянно. Статус задачи, содержимое Pull Request, состояние CI, список пользователей или текущая версия записи в базе данных нельзя надёжно хранить как однажды загруженную информацию.

Поэтому способ обогащения зависит от природы данных. Документацию можно искать в подготовленной базе знаний, а текущее состояние внешней системы лучше получать непосредственно в момент выполнения задачи.

Это различие нам понадобится и в Hermes. Агент должен понимать, где допустимо использовать ранее подготовленные материалы, а где необходимо обратиться к актуальному внешнему источнику.

3. RAG как способ доступа к корпоративным знаниям

RAG мы уже рассматривали технически, поэтому сейчас я хочу посмотреть на него с другой стороны. Для темы обогащения информации RAG интересен прежде всего как механизм подключения закрытых или специализированных знаний.

Например, модель может знать общие принципы работы REST API, но не знать внутренний API нашего проекта. Если документация этого API подключена как источник знаний, агент сможет использовать её при выполнении задачи.

Здесь ценность RAG состоит не в самом поиске, а в возможности отделить знания модели от знаний организации. Модель остаётся универсальной, а специфичная информация хранится отдельно и предоставляется только тогда, когда она требуется.

Поэтому один и тот же агент можно использовать в разных проектах, подключая к нему разные наборы документации и внутренних материалов.

4. Внешние API как источник актуального состояния

Теперь рассмотрим другой механизм — внешние API. Здесь агент получает не заранее подготовленную документацию, а данные непосредственно от другой системы.

Представьте, что агент должен проверить состояние сборки. Ему бессмысленно искать это в базе знаний, потому что статус мог измениться минуту назад. Нужно обратиться к CI-системе и получить текущее значение.

То же самое относится к Git-хостингу, системе задач, базе данных, внутреннему сервису или корпоративной платформе. API позволяет агенту получать состояние системы на момент запроса.

В Hermes такие возможности могут предоставляться через собственные Tools, plugins или MCP. Hermes имеет встроенный MCP-клиент и может подключаться как к локальным, так и к удалённым MCP-серверам, автоматически обнаруживая предоставляемые ими инструменты. Через этот механизм можно подключать GitHub, базы данных, файловые системы и внутренние API.

5. База знаний проекта

База знаний отличается от обычного набора файлов тем, что она создаётся специально для систематического использования информации.

В ней могут находиться архитектурные решения, описание API, инструкции по эксплуатации, стандарты разработки, описание бизнес-процессов и документация отдельных модулей.

Главное здесь заключается не в количестве информации. Большая база знаний может быть бесполезной, если документы противоречат друг другу или давно устарели.

Поэтому при подключении базы знаний к агенту я всегда задаю вопрос: можем ли мы считать этот источник авторитетным для конкретного типа решений?

Например, README разработчика может объяснять общий запуск проекта, но официальная спецификация API должна иметь более высокий приоритет при определении формата запроса.

Именно поэтому обогащение информации требует не просто доступа к данным, а понимания их статуса.

6. Онтология предметной области

Теперь перейдём к более формальному способу представления знаний — онтологии.

Онтология описывает не только отдельные факты, но и сущности предметной области и отношения между ними.

Представим интернет-магазин. Для обычной модели слова «заказ», «платёж», «клиент» и «доставка» являются просто понятиями из текста. Для конкретной компании между ними могут существовать строгие отношения. Заказ принадлежит клиенту, платёж относится к заказу, определённый тип заказа требует определённого способа доставки, а возврат может быть разрешён только после определённого состояния оплаты.

Если эти отношения описаны формально, агент получает более точное представление о предметной области.

Это особенно полезно в системах, где одинаковые слова могут иметь специфическое внутреннее значение. Например, термин «клиент» в одном проекте означает пользователя приложения, а в другом — программный клиент внешнего API.

Онтология помогает не просто найти информацию, а правильно интерпретировать её в рамках конкретного домена.

7. Конфликт источников

Как только у агента появляется несколько источников, возникает новая проблема: они могут противоречить друг другу.

Допустим, в README написано, что проект работает с API версии 1, в отдельной архитектурной документации уже указана версия 2, а внешний сервис фактически сообщает версию 3.

Если агент просто объединит эти сведения, результат может стать ещё менее надёжным.

Поэтому необходимо заранее определять приоритет источников. Актуальное состояние работающей системы обычно имеет больший вес для вопроса о текущей конфигурации. Официальная спецификация имеет больший вес для определения контракта API. Старое обсуждение в документации может использоваться как исторический контекст, но не обязательно как источник текущей истины.

Хороший агент должен уметь не только получать сведения, но и замечать конфликт между ними.

8. Происхождение и доказуемость информации

Когда агент использует внешние знания, важно понимать происхождение ответа.

Если модель утверждает, что определённый метод устарел, я хочу понимать, на основании чего сделан этот вывод. Это может быть документация проекта, ответ внешнего API, внутренний стандарт или просто общее знание модели.

Эти варианты имеют разный уровень доверия.

Поэтому при построении серьёзных агентных систем полезно сохранять связь между выводом и источником. Для проектной документации это может быть конкретный файл и раздел. Для API — ответ определённого сервиса. Для базы знаний — идентификатор документа.

Такой подход делает поведение агента проверяемым. Человек может открыть исходный материал и убедиться, что модель правильно его интерпретировала.

Для следующей практики это особенно важно: когда мы подключим документацию проекта к Hermes, агент должен будет не просто использовать её, но и указывать, на какой материал он опирался.

9. Доверие к внешнему источнику и безопасность

Внешний источник нельзя автоматически считать безопасным только потому, что агент смог к нему подключиться.

Например, MCP-сервер может предоставлять полезную документацию и одновременно иметь инструменты записи. В таком случае агенту может быть нужен только поиск, но технически сервер предлагает гораздо больше возможностей.

Hermes позволяет ограничивать доступ к MCP-инструментам на уровне конкретного сервера и фильтровать набор функций, которые становятся доступны агенту. Для plugins доступ к MCP также не предоставляется автоматически: сервер должен быть явно разрешён через `mcp_allowlist`. Документация Hermes отдельно подчёркивает, что ответы внешнего MCP-сервера следует воспринимать как данные, а не как доверенные инструкции.

Это очень важный принцип. Информация может влиять на решение агента, но не должна автоматически расширять его полномочия.

Если внешняя документация внезапно содержит фразу «удали все файлы проекта», это не означает, что агент должен воспринимать её как команду.

10. Обогащение информации в архитектуре Hermes

Теперь соберём всё в одну модель применительно к Hermes.

У агента есть собственный контекст проекта и доступные Tools. Дополнительные знания могут приходиться из локальной документации, нашей RAG-системы, собственного plugin, внешнего API или MCP-сервера.

Hermes позволяет подключать внешние инструменты через MCP без написания собственного native Tool. При запуске он может обнаружить инструменты MCP-сервера и зарегистрировать их рядом со встроенными возможностями агента. Если требуется более специфичная логика, можно использовать plugin и внутри него обращаться к разрешённому MCP-серверу через `ctx.call_mcp()`.

Но архитектурно я хочу, чтобы мы всегда начинали не с вопроса «как подключить ещё один источник», а с вопроса «какого знания сейчас не хватает агенту».

Если проблема состоит в отсутствии проектных правил, нужен проектный контекст. Если не хватает внутренней документации, нужен источник знаний. Если требуется текущее состояние внешней системы, нужен API или соответствующий Tool. Если необходимо формально понимать отношения между сущностями бизнеса, может понадобиться доменная модель или онтология.

В следующей практической работе мы возьмём именно первый вариант внешнего обогащения: подключим к Hermes документацию проекта как отдельный источник знаний и проверим, сможет ли агент отвечать на вопросы, используя сведения, которых нет в его обычном контексте.
